Le type `Array`

Définition

Un tableau est une structure de données qui permet de stocker une liste d'éléments. Chaque élément est identifié par un indice. En TypeScript, les indices commence à 0. Les tableaux sont homogènes, c'est-à-dire qu'ils ne peuvent contenir que des éléments du même type.

En TypeScript, les tableaux sont définis par le type `Array` qui est un type générique. Le type générique est défini par le type des éléments qu'il contient. Par exemple, le type `Array<number>` est un tableau qui contient des nombres.

Valeurs littérales

En TypeScript, la valeur littérales d'un tableau est définie par ses éléments listés entre crochets et séparés par des virgules. Par exemple, le tableau `[1, 2, 3]` est un tableau qui contient les nombres 1, 2 et 3.

Déclaration d'une variable de type tableau à partir d'une valeur littérale

Si on connaît la valeur littérale d'un tableau, on peut déclarer une variable de type tableau en utilisant le type générique `Array` :

```
let tab = [1, 2, 3];
```

Le type de la variable `tab` est `Array<number>`. Le type est déduit automatiquement à partir de la valeur littérale. Par exemple, on peut déclarer un tableau de chaînes de caractères :

```
let tab = ["a", "b", "c"];
```

Le type de la variable `tab` est `Array<string>`.

Déclaration d'une variable de type tableau sans valeur littérale

Si on ne connaît pas la valeur littérale d'un tableau, on peut déclarer une variable de type tableau en utilisant le type générique `Array` :

```
let tabNumber: Array<number>;
let tabString: Array<string>;
let tabBoolean: Array<boolean>;
let tabArrayNumber: Array<Array<number>>;
```

Remarque : *générique* signifie que le type tableau peut contenir n'importe quel type d'élément. Par contre, quand on définit un tableau, on doit préciser le type des éléments qu'il contient.

Ce type de déclaration est aussi celui à utiliser pour les paramètres de fonctions qui attendent un tableau en paramètre ainsi que pour le type de retour d'une fonction.

```
function copie(t: Array<number>): Array<number> {
    ...
}
```

Allocation d'un tableau

Si on ne donne pas de valeur littérale à un tableau, il faut l'allouer avant de pouvoir l'utiliser. Pour allouer un tableau, on utilise l'opérateur `new` :

```
let tabNumber = new Array<number>();
let tabString = new Array<string>();
```

Lorsque l'on alloue un tableau de cette manière, il est vide. Il faut ensuite ajouter des éléments au tableau.

Accès aux éléments d'un tableau

Pour accéder à un élément d'un tableau, on utilise l'opérateur `[]` :

```
let tab = [1, 2, 3];
console.log(tab[0]); // Affiche 1
console.log(tab[1]); // Affiche 2
console.log(tab[2]); // Affiche 3
```

En TypeScript, les indices des éléments d'un tableau commencent à 0. L'indice du premier élément est 0, celui du second élément est 1, etc.

Rappels : toute expression entière peut être utilisée comme indice d'un tableau. Par exemple, on peut utiliser une variable :

```
let tab = [1, 2, 3];
let i = 1;
console.log(tab[i]); // Affiche 2
console.log(tab[i + 1]); // Affiche 3
```

Taille d'un tableau

Pour connaître la taille d'un tableau, on utilise la propriété `length` :

```
let tab = [1, 2, 3];
console.log(tab.length); // Affiche 3
```

Parcours d'un tableau

Pour parcourir un tableau, on utilise une boucle `for` :

```
let tab = [1, 2, 3];

for (let i = 0; i < tab.length; i++) {
  console.log(tab[i]);
}
```

Il est aussi possible d'utiliser une boucle `for..of` qui permet de parcourir les éléments d'un tableau :

```
let tab = [1, 2, 3];

for (let e of tab) {
  console.log(e);
}
```

```
1
2
3
```

On utilise la boucle `for..of` pour parcourir les éléments d'un tableau. On utilise la boucle `for` pour parcourir les indices d'un tableau. On peut aussi utiliser la boucle `for..in` pour parcourir les indices d'un tableau :

```
let tab = [1, 2, 3];

for (let i in tab) {
  console.log("tab[" + i + "] = " + tab[i]);
}
```

```
tab[0] = 1
tab[1] = 2
tab[2] = 3
```

Remarque : la variable dans une boucle `for` est de type `string`. De ce fait, si on veut utiliser cette variable dans une expression arithmétique, il faut la convertir en nombre avec la fonction `parseInt` :

```
let tab = [1, 2, 3];

for (let i in tab) {
  console.log("tab[" + i + "] = " + tab[i]);
  console.log("tab[" + i + "] + 1 = " + (parseInt(i) + 1));
}
```

```
tab[0] = 1
tab[0] + 1 = 2
tab[1] = 2
tab[1] + 1 = 3
tab[2] = 3
tab[2] + 1 = 4
```

Mutabilité des tableaux

A la différence des chaînes de caractères, il est possible de modifier un élément dans un tableau.

```
let tab = [1, 2, 3, 4, 5];

for (let i = 0; i < tab.length; i++) tab[i] = 2 * tab[i] + 1;

console.log(tab);
```

```
[3, 5, 7, 9, 11]
```

Validité des indices

Attention : TypeScript ne vérifie pas la validité d'un indice

1. si l'indice n'est pas un entier ou est strictement négatif, l'accès retourne la valeur `undefined`

```
let tab = [1, 2, 3, 4, 5];
console.log(tab[1.5]);
```

```
undefined
```

2. si l'indice n'est pas un entier ou est strictement négatif, l'affectation fait *quelque chose* (mais il est peu probable que ce soit ce que vous vouliez)

```
let tab = [1, 2, 3, 4, 5];
tab[1.5] = 42;
tab[-1] = -42;
console.log(tab);
```

```
[ 1, 2, 3, 4, 5, '1.5': 42, '-1': -42 ]
```

3. Si l'indice est bien un entier entre 0 et la longueur du tableau - 1, l'accès retourne la valeur de l'élément et l'affectation modifie l'élément

```
let tab = [1, 2, 3, 4, 5];
tab[2] = 42;
let x = 2 * tab[2];

console.log("tab =", tab);
console.log("x =", x);
```

```
tab = [ 1, 2, 42, 4, 5 ]
x = 84
```

4. si l'indice est supérieur ou égal à la longueur du tableau :
    1. la taille du tableau est modifiée pour être égale à l'indice plus 1
    2. les nouveaux éléments reçoivent la valeur 10

```
let tab = [1, 2, 3, 4, 5];
tab[8] = 42;

for (let i = 0; i < tab.length; i++) console.log("tab[" + i + "] = " + tab[i]);
```

```
tab[0] = 1
tab[1] = 2
tab[2] = 3
tab[3] = 4
tab[4] = 5
tab[5] = undefined
tab[6] = undefined
tab[7] = undefined
tab[8] = 42
```

Ajout d'un élément à la fin d'un tableau

Un tableau est un **objet** $\approx$ des données + des fonctions pour accéder / modifier. L'utilisation d'une méthode se fait par une notation `.` sur une variable de type tableau.

Les tableaux de type `Array` sont des tableaux dynamiques. On peut ajouter des éléments à la fin d'un tableau en utilisant la méthode `push` :

Par exemple, le programme suivant construit le tableau contenant les entiers pairs plus petits strictement que `n`.

```
let n = 10;

let tab = new Array<number>();

for (let i = 0; i < n; i += 2) {
  console.log("Itération pour i =", i);
  console.log("tab avant push =", tab);
  tab.push(i);
  console.log("tab après push =", tab);
  console.log();
}

console.log("tab après la boucle");
console.log(tab);
```

```
Itération pour i = 0
tab avant push = []
tab après push = [ 0 ]

Itération pour i = 2
tab avant push = [ 0 ]
tab après push = [ 0, 2 ]

Itération pour i = 4
tab avant push = [ 0, 2 ]
tab après push = [ 0, 2, 4 ]

Itération pour i = 6
tab avant push = [ 0, 2, 4 ]
tab après push = [ 0, 2, 4, 6 ]

Itération pour i = 8
tab avant push = [ 0, 2, 4, 6 ]
tab après push = [ 0, 2, 4, 6, 8 ]

tab après la boucle
[ 0, 2, 4, 6, 8 ]
```

Retirer un élément à la fin d'un tableau

La méthode `pop` permet de retirer le dernier élément d'un tableau. La méthode retourne l'élément qui vient d'être retiré.

```
let tab = [0, 2, 4, 8];

while (tab.length > 0) {
  console.log("tab avant pop =", tab);
  console.log("élément retiré =", tab.pop());
  console.log("tab après pop =", tab);
  console.log();
}
```

```
tab avant pop = [ 0, 2, 4, 8 ]
élément retiré = 8
tab après pop = [ 0, 2, 4 ]

tab avant pop = [ 0, 2, 4 ]
élément retiré = 4
tab après pop = [ 0, 2 ]

tab avant pop = [ 0, 2 ]
élément retiré = 2
tab après pop = [ 0 ]

tab avant pop = [ 0 ]
élément retiré = 0
tab après pop = []
```

Appliquer `pop` sur un tableau vide ne cause pas d'erreur et retourne la valeur `undefined`.

```
let tab = new Array<number>(); // tab est un tableau vide

console.log("on tente de retirer un élément =", tab.pop());
```

```
on tente de retirer un élément = undefined
```

Aliasing

Question : que se passe-t'il quand on affecte une variable de type tableau à une autre variable de type tableau ?

Considérons le programme suivant :

```
let t1 = [1, 2, 3, 4, 5];
let t2 = t1;

console.log(t1);
console.log(t2);
```

```
[ 1, 2, 3, 4, 5 ]
[ 1, 2, 3, 4, 5 ]
```

Question : quel est le rapport entre `t1` et `t2` ?

- **Possibilité 1 :** `t1` et `t2` font référence à deux tableaux distincts qui ont la même valeur (la séquence `1, 2, 3, 4, 5`)
- **Possibilité 2 :** `t1` et `t2` font référence au même tableau, ce qui signifie que modifier `l1` entraîne une modification de `l2` et vice versa.

Faisons un test :

```
let t1 = [1, 2, 3, 4, 5];
let t2 = t1;

t2[2] = 42;

console.log(t1);
console.log(t2);
```

```
[ 1, 2, 42, 4, 5 ]
[ 1, 2, 42, 4, 5 ]
```

On est donc dans le second cas.

L'association d'une variable avec un tableau est appelée une **référence**.

Un même tableau peut être donc être référencé par plusieurs variables.

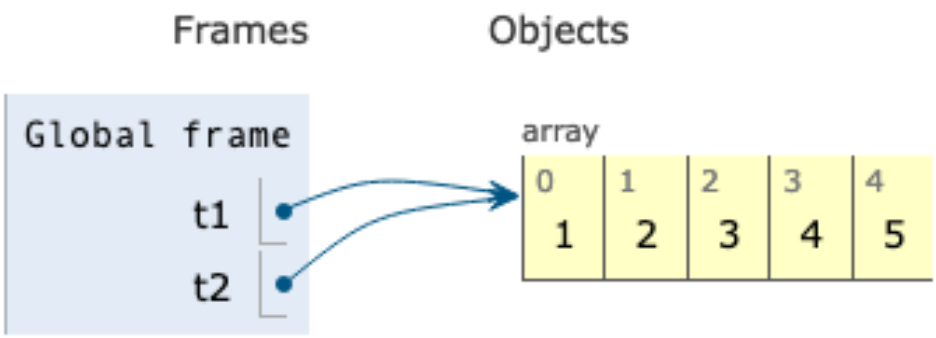
La modification d'un tableau via une variable qui le référence est donc visible via toutes les autres variables qui le référencent.

Cet effet est appelé **aliasing**.

On peut illustrer cela du point de la mémoire. Considérons le code :

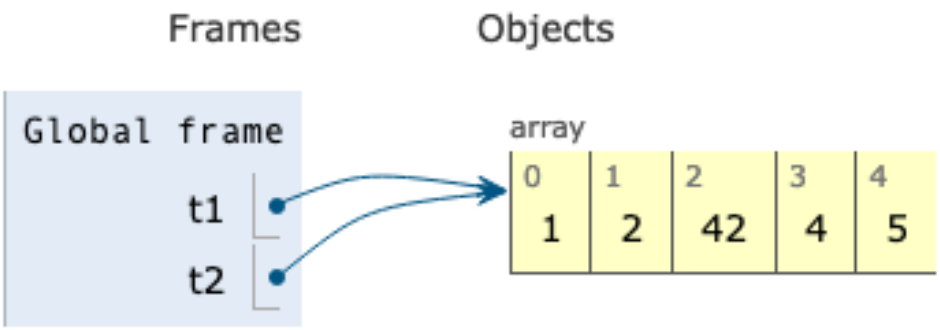
```
let t1 = [1, 2, 3, 4, 5];
let t2 = t1;
```

En mémoire, `t1` et `t2` font bien référence au même tableau :



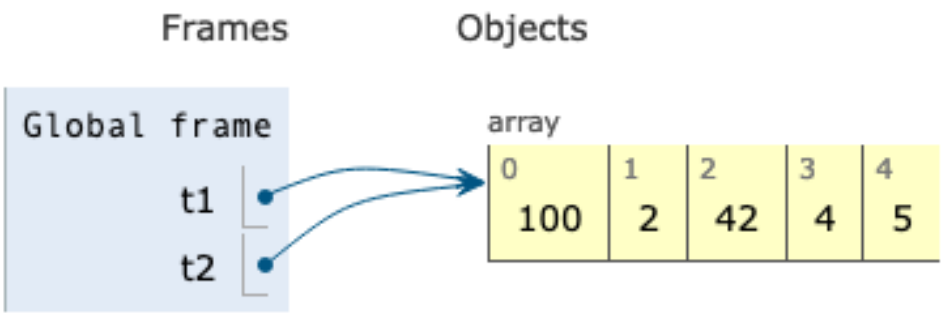
Si on modifie `t2`, c'est bien le tableau partagé en mémoire que l'on modifie :

```
t2[2] = 42;
```



De même, si on modifie `t1` :

```
t1[0] = 100;
```



Copie profonde

Question : comment faire une copie d'un tableau qui crée un nouveau tableau ? (c'est que l'on appelle une **copie profonde**)

Solution 1 : écrire sa propre fonction

```
function copie(t: Array<number>): Array<number> {
  let nouveau: Array<number> = new Array<number>();

  for (let valeur of t) nouveau.push(valeur);

  return nouveau;
}

let t1 = [1, 2, 3, 4, 5];
let t2 = copie(t1);
```

Solution 2 (meilleure) : utiliser l'opérateur **spread** `...`

```
let t1 = [1, 2, 3, 4, 5];
let t2 = [...t1];
```

Comparaison de tableaux

Considérons les instructions suivantes :

```
let t1: Array<number> = [1, 2, 3, 4, 5];
let t2: Array<number> = [1, 2, 3, 4, 5];

console.log(t1 === t2);
```

Question : Qu'est-ce qui est testée ? Les références ou la séquence ?

Regardons l'affichage :

```
false
```

On teste donc les références et pas la valeur

Pour tester si deux tableaux ont la même séquence, il faut écrire une fonction ad-hoc :

```
function égal(t1: Array<number>, t2: Array<number>): boolean {
  if (a === b) return true;
  if (a == null || b == null) return false;
  if (t1.length != t2.length) return false;

  for (let i in t1) if (t1[i] !== t2[i]) return false;
```

```
  return true;
}
```

Tableau et fonction

Lorsqu'un tableau est passé en argument d'une fonction, on passe la référence sur l'objet et non une copie.

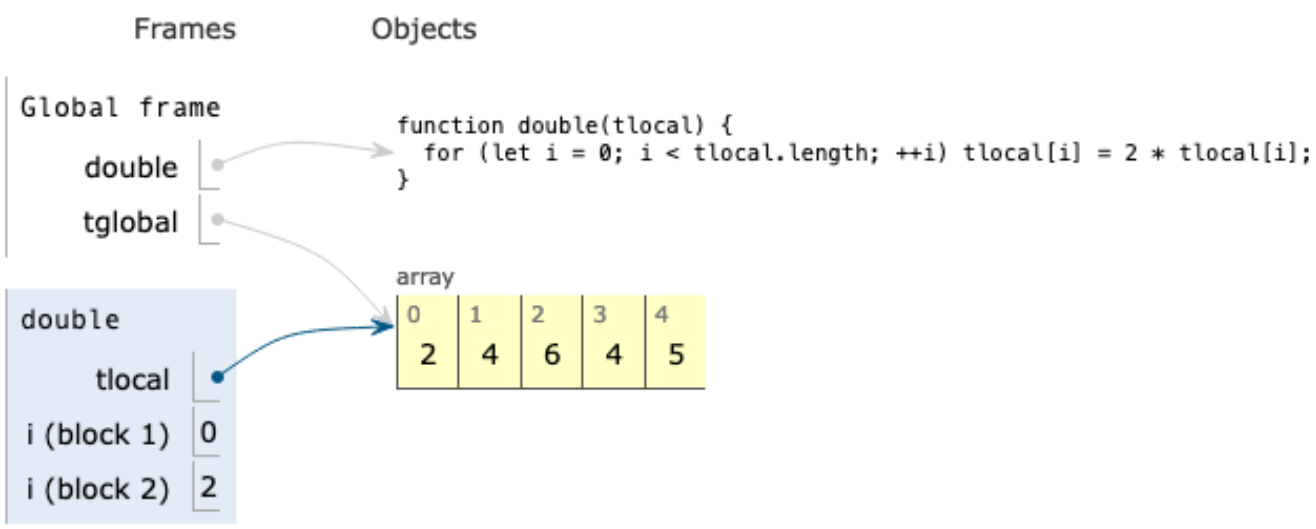
Conséquence : si la fonction modifie le tableau, la modification est visible au niveau de l'appel.

```
function double(tlocal: Array<number>): void {
  for (let i in tlocal) tlocal[i] = 2 * tlocal[i];
}

let tglobal = [1, 2, 3, 4, 5];
double(tglobal);
```

```
[ 2, 4, 6, 8, 10 ]
```

Voici ce qui se passe du point de vue de la mémoire :



Une fonction peut très bien avoir un tableau en type de retour. Il faut allouer (créer) le tableau dans la fonction 😒

```
function créeTableauAléatoire(n: number): Array<number> {
  let t = new Array<number>();

  for (let i = 0; i < n; i++) t.push(Math.random());

  return t;
}
```

```
let nouveauTableaAléatoire = créeTableauAléatoire(10);
```